Tecsup VIRTUAL

BOOTCAMP FRONTEND & REACT · SEMANA 4

El contrato

Clase 12 — El proyecto tipado: TypeScript aplicado a TechCart. Tipos, interfaces, aduanas y el puente a React.

Último día de fundamentos Proyecto TechCart

MASTERCLASS

ÍNDICE

HOY LA TIENDA NO GANA UN SOLO BOTÓN. GANA UN CONTRATO.

Agenda

1. **Tres veces lo mismo, y nadie avisó**
   por qué existe TypeScript · tipado dinámico y estático
2. **Las herramientas**
   Node, npm, `tsc`, `tsconfig` y el modo estricto
3. **Tipos básicos e inferencia**
   y la pareja del día: any frente a unknown
4. **El contrato: `tipos.ts`**
   interfaces, uniones, opcionales, `readonly`, guardas
5. **Primera aduana: la API**
   el adaptador, dicho con tipos
6. **Segunda aduana: el navegador**
   el respaldo y el carrito guardado
7. **El DOM, y un genérico propio**
   un elemento que puede no estar
8. **El checkout migrado + cierre**
   la tercera aduana, errores frecuentes y 5 tareas

☕ **Receso de 15 min**, fijo · verde = **el archivo nuevo del día** · naranja = **cierre del bloque de fundamentos**

POR QUÉ

LOS TRES PASARON LA CLASE PASADA, EN VIVO

Los tres eran el mismo error

`precio × nada`

el carrito guardado no traía `cantidad`, y el subtotal lo multiplicó igual.

`marca vacía`

si la marca llegaba sin valor, el buscador moría en la **primera tecla**.

`"78" ≠ 78`

lo que sale de un atributo del HTML es **texto**: compararlo con un número da falso.

Un dato que no tenía la forma que el código esperaba. Y en los tres, **nadie avisó**.

> Y un cuarto, el más claro: escribí `p.title` para leer el nombre del producto. **¿Y si me equivoco y escribo `p.titel`?** No pasa nada. Ni un aviso, ni un subrayado, ni un error al correr: la tienda pinta un vacío en 38 tarjetas.

POR QUÉ

LA PREGUNTA QUE ABRE LA CLASE

¿Dónde está escrito qué es un producto?



Hoy me acordé de las cuatro. La próxima vez no me voy a acordar. Y en React, un producto lo reciben **veinte componentes**.

EL MODELO MENTAL DE TODO EL DÍA

TypeScript no se ejecuta: se compila

lo que escribimos · .ts

```ts
interface Producto {
  id: number
  nombre: string
}

const formatear = (p: Producto) =>
  `${p.nombre}`
```

→ tsc

lo que ve el navegador · .js

```js
// la interface: desapareció entera


const formatear = (p) =>
  `${p.nombre}`
```

⚠️ **Los tipos se borran al compilar.** De ahí salen las dos consecuencias del día: **(1)** TypeScript no hace **nada** mientras el programa corre: no valida la API, no revisa lo guardado; **(2)** sirve para que el editor avise **antes de correr**. **Te protege de tus propios errores, no de los datos que llegan de fuera.**

JAVA

LA DIFERENCIA QUE MÁS LOS VA A DESCOLOCAR

Java mira el nombre. TypeScript mira la forma.

JAVA · TIPADO NOMINAL

```
class Producto implements Vendible {…}

// si no lo DECLARA, no lo es:
// aunque tenga los mismos campos
```

Manda el **nombre**.

VS

TYPESCRIPT · TIPADO ESTRUCTURAL

```
const suelto = { x: 5, y: 9, color:
"rojo" }

const b: Punto = suelto  // ✅ entra
// nunca dijo ser un Punto… pero TIENE
la forma
```

Manda la **forma**.

¿Por qué así? Porque en JavaScript los objetos **vienen de fuera**: de un servidor, de lo guardado, de un formulario. Ninguno trae una etiqueta con su clase. Lo único que tienen es una **forma**.

NODE

LA PRIMERA CLASE DEL CURSO QUE INSTALA ALGO

El navegador no entiende TypeScript

Node.js
ejecuta JavaScript **fuera** del navegador. Es quien hace correr al compilador.

+

npm
viene en el mismo instalador. Baja las herramientas **a la carpeta del proyecto** y deja anotado qué bajó.

LO QUE YA CONOCEN

Node es a JavaScript lo que la **JVM** es a Java. `npm` es su **Maven**, `package.json` es el `pom.xml` y `node_modules` es la carpeta de dependencias, pero **por proyecto**.

AL INSTALAR

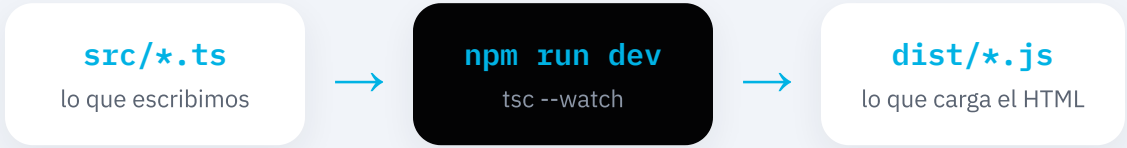
Bajen la versión **LTS**, nunca la "Current". Valores por defecto a todo. Y cuando termine: **cierren la terminal y VS Code y ábranlos de nuevo**, o les va a seguir diciendo que node no existe.

`node_modules` **no se sube a git**: está en el `.gitignore` desde la clase 1 y se recupera con `npm install`.

LOS MISMOS TRES DE TODOS LOS PROYECTOS QUE HAGAN

Tres comandos

`npm init -y`	crea **package.json**: la ficha del proyecto y **de qué depende**
`npm i -D typescript`	baja TypeScript **a este proyecto**. El `-D` = "para desarrollar, no para publicar": el navegador nunca lo ve
`npx tsc --init`	crea **tsconfig.json**. Lo reemplazamos entero: `rootDir: src`, `outDir: dist`, `lib: DOM` y `strict: true`



`strict` no es una opción: son como diez juntas. Las dos que más se notan: `noImplicitAny` ("esta función recibe algo y nadie dijo qué es") y `strictNullChecks` ("esto puede no existir: no lo uses sin preguntar"). Apagarlas es quedarse sin el motivo para usar TypeScript.

Todo JavaScript válido es TypeScript válido. Se renombra `.js` → `.ts`, se cambia **una línea** del HTML, y los tipos se agregan de a poco.

VIENEN DE JAVA: ESTO YA LO SABEN

No anoten lo obvio

```
string  // en minúscula, no String
number  // UNO solo: no int/long/double
boolean
null  ·  undefined  // aquí SÍ son tipos
Producto[]  // un arreglo de productos
```

```
const moneda: string = "$"  // de más
const moneda = "$"  // ✅ ya sabe que es string

// se anota lo que NO puede adivinar:
const formatear = (p: number): string =>
  `${MONEDA}${p.toFixed(2)}`
```

Se llama **inferencia**. Si su código se llena de `: string`, están anotando de más: los **parámetros** sí, el resto se deduce.

Y el retorno es opcional: se pone en las funciones exportadas, porque **documenta** y porque si rompes la función el aviso sale **ahí**, no en los cinco sitios que la llaman.

ANY

LA PAREJA QUE DECIDE LA SEGUNDA MITAD DE LA CLASE

any apaga TypeScript. unknown obliga a mirar.

any

```
const x: any = "un texto"
x.toFixed(2) // ✅ compila… 💥 falla al correr
```

"apaga TypeScript aquí". Acepta cualquier cosa y deja hacerle cualquier cosa. Es un **agujero** en el contrato, y no se nota porque no da aviso.

unknown

```
const x: unknown = "un texto"
x.toFixed(2) // ❌ el editor frena
if (typeof x === "number") { … } // ✅
```

"no sé qué es" (igual que any) "**pero no lo tocas hasta que me demuestres qué es**".

any es una decisión, no un descuido. Si no sabes qué tipo poner, la respuesta casi siempre es unknown. Al terminar la clase este proyecto tiene **cero any**.

LA RESPUESTA A LA PREGUNTA DE HACE UN RATO

Qué es un producto: en un solo sitio

api.ts

ui.ts

tienda.ts

carrito.ts

→

```ts
export interface Producto {
  readonly id: number
  nombre: string
  marca: string
  precio: number
  categoria: Categoria
  stock: number
  destacado: boolean
  imagen: string
  alt: string
}
```

`tipos.ts` · y al compilar **desaparece entero**

Nueve campos, **todos obligatorios**: si algo tiene esta forma, el resto del proyecto puede darlo por hecho sin preguntar. Y `categoria` no es texto suelto: es **una de las cuatro que vendemos**.

TODO LO DE ESTA LÁMINA REAPARECE EN REACT

El | se lee "o". El & se lee "y".

```ts
type Categoria = "laptops" | "smartphones" | …
type Filtro = Categoria | "todas"

// un producto Y ADEMÁS una cantidad
type LineaCarrito =
  Producto & { cantidad: number }

// ? = puede no estar · readonly = no se reasigna
  brand?: string
  readonly id: number
```

```ts
// una FUNCIÓN también tiene tipo
(precio: number) => string
(carrito: LineaCarrito[]) => void

// void = "no devuelve nada útil":
// existe para HACER, no para calcular
export const guardarCarrito =
  (c: LineaCarrito[]): void => {…}
```

JAVA

Un enum crea una **clase** que existe cuando el programa corre. Y un parámetro "número o texto" no se puede escribir.

TYPESCRIPT

La unión **no crea nada**: al compilar queda el texto pelado. Toda la seguridad, cero peso. Y `number | string` se escribe y ya está.

`interface` para las formas de objeto, `type` para todo lo demás. Esa es la regla y no hay que pensarla más.

ESTRECHAR

LO MÁS ELEGANTE DE TYPESCRIPT

El compilador sigue tus if

```ts
const copia: unknown = JSON.parse(crudo)
// aquí: unknown. No se puede tocar nada.

if (typeof copia !== "object" || copia === null)
return null
// aquí: ya es un objeto

if (!("fecha" in copia) || typeof copia.fecha !==
"string") return null
// aquí: copia.fecha ya es string ✅
```

Ninguno es una ceremonia nueva: son los mismos `if` que escribirías igual en JavaScript. La diferencia es que el compilador los lee y lleva la cuenta.

`typeof`
para los tipos básicos

`Array.isArray`
para saber si es una lista

`instanceof`
¿es de esta clase? `error instanceof Error`, `evento.target instanceof Element`

`valor is Categoria`
tu propia guarda: una función normal cuyo retorno enseña al compilador a estrechar. Hoy escribimos **tres**.

POR AQUÍ ENTRA TODO LO QUE NO ESCRIBIMOS NOSOTROS

Anotar un tipo no valida nada

🌐

La API

`respuesta.json() → any`

✅ aduana: el adaptador

💾

El navegador

`JSON.parse() → any`

✅ aduana: unknown + isArray

⌨️

El formulario

`string | File`

✅ aduana: leerCliente()

⚠️ **TypeScript no revisa ninguna de las tres mientras el programa corre**: los tipos ya se borraron. Si escribes `const p: Producto[] = await r.json()`, el compilador se queda **contento**... y si el servidor manda otra cosa, falla igual que la semana pasada. **La anotación es un deseo; la aduana es la que lo cumple.**

Y la segunda puerta no la abre un extraño: **lo guardado lo escribió una versión anterior de tu propio programa**.

EL ERROR MÁS COMÚN DE JAVASCRIPT EN EL NAVEGADOR, MARCADO ANTES DE ABRIRLO

Object is possibly 'null'

```
// 1 · el "!": le prometo al compilador que existe
const c = document.querySelector("#x")!
// no comprueba NADA: apaga el aviso
```

```
// 2 · un if por elemento: honesto, y son TRECE
if (!c) throw new Error(…)
```

```
// 3 · ✅ una función que lo comprueba UNA vez
const pedirElemento =
  <T extends Element>(sel: string): T => {…}
```

y resuelve un segundo problema

un `Element` pelado **no tiene** `.value`, ni `.disabled`, ni `.style`, ni `.dataset`. Los tienen los campos, los botones y las cajas.

```
pedirElemento<HTMLInputElement>("#buscador").value
pedirElemento<HTMLDivElement>("#barra-envio").style
```

⚠️ **El `!` no es una solución.** Alguien lo pone un viernes para salir del paso; seis meses después cambia el orden de carga y falla en producción, **sin ningún aviso del compilador**, porque alguien le prometió que no pasaría.

ESTO YA LO SABEN DE JAVA: LIST<STRING>

Un tipo que recibe otro tipo

`Producto[]`

un arreglo **de productos** · es lo mismo que `Array<Producto>`

`Promise<Producto[]>`

una promesa **de** un arreglo de productos · el `await` la desenvuelve sola

`Record<Categoria, string>`

una clave por cada categoría · si falta una, **no compila**

```
const pedirElemento = <T extends Element>(
  selector: string,
): T => {
  const el = document.querySelector<T>(selector)
  if (!el) throw new Error(…)
  return el
}
```

`<T>`	**declara** el tipo que recibe
`extends Element`	le pone un **límite**
`: T`	devuelve **lo mismo que le pidieron**

Un genérico **conserva** la información del tipo en vez de aplastarla: **quien llama decide qué recibe**. Y al compilar, como todo lo demás, **no queda nada**.

FUNCIONA DESDE LA CLASE PASADA · HOY NO SE REESCRIBE, SE TIPA

La tercera puerta estaba abierta y no la vimos

```
// esta línea existe desde la clase pasada y funciona
Object.fromEntries(new FormData(formulario))

// …y el editor dice que devuelve:
{ [k: string]: FormDataEntryValue }
// = string | File, porque un form PUEDE tener archivos
```

TypeScript **no tiene idea** de que ahí dentro hay un nombre, un correo y un método de pago. Y tiene razón: eso depende del HTML.

```
// la salida honesta: una aduana que no puede mentir
const leerCliente = (f: HTMLFormElement): Cliente => ({
  nombre: textoDe(datos, "nombre"),
  …
  pago: esMetodoPago(datos.pago) ? datos.pago : "tarjeta"
})
```

⚠️ La salida fácil es `as Cliente`. Y el compilador contesta: **"neither type sufficiently overlaps... convert the expression to 'unknown' first"**. Dos conversiones encadenadas son una señal: **alguien se saltó una comprobación**.

Un formulario **no tiene números**: todo lo que sale de él es texto. Los nueve campos de `Cliente` son `string`, y el método de pago es la **unión de los tres valores** que están en el HTML desde la clase 1.

TIPADO

LA MISMA TIENDA DE AYER · Y EN PANTALLA NO SE MOVIÓ UN PÍXEL

Nueve archivos, un contrato

`tipos.ts`
qué ES cada cosa

`datos.ts`
qué vende la tienda

`formato.ts`
cómo se ve un valor

`tienda.ts`
cuentas y filtros

`carrito.ts`
las reglas del carrito

`respaldo.ts`
la copia local

`api.ts`
habla y traduce

`ui.ts`
cómo se ve una cosa

`main.ts`
orquesta y escucha

`tipos.ts` **no genera una sola línea** de JavaScript… y sostiene a los otros ocho.

`0 KB en el navegador`
la única mejora de calidad de código que no cuesta rendimiento

`npm run check` (por dentro, `tsc --noEmit`) revisa el proyecto entero **sin escribir un archivo**. Eso corre antes de cada entrega.

ERRORES

LOS DOS PRIMEROS NO DAN NINGÚN AVISO. POR ESO SON LOS PEORES.

Errores frecuentes

1 · callar con any	sale un subrayado, no se entiende, se le pone `any` y se va. Con él se fue TypeScript: `p.titel` **compila**. Primos del mismo vicio: el `!` y el `as` sin comprobar.
2 · confiar en la API	`const p: Producto[] = await r.json()` **compila sin una queja** y falla igual que antes. La anotación es un deseo; lo único que lo cumple es la **aduana**.
3 · el valor es texto	todo lo que sale del HTML es cadena. El mensaje no es el que esperas: **"This comparison appears to be unintentional because the types 'string | undefined' and 'number' have no overlap"**.
4 · creer que valida	no existe "comprobar si esto es un `Producto`" al correr, porque **`Producto` no existe al correr**. Lo único que existe son los `if` que escribiste.
5 · el DOM que no está	este sí lo grita: **"Object is possibly 'null'"**. El tropiezo no es no verlo: es taparlo con un `!`. Trece elementos son trece promesas sin comprobar.

CIERRE

SEIS IDEAS, EN EL ORDEN EN QUE LAS VIMOS

Resumen

`los tipos se borran`
es un compilador: el navegador recibe JavaScript

`interface = la forma`
estructural, no nominal: la gran diferencia con Java

`| ? & readonly`
uniones, opcionales, intersecciones y solo lectura

`estrechar`
el compilador lee tus `if`; y tú escribes tus guardas

`any frente a unknown`
apagar TypeScript, o que te obligue a mirar

`las tres aduanas`
la API · el navegador · el formulario

Anotar un tipo no valida nada.
TypeScript te cuida de tus propios errores, **no de los datos que llegan de fuera**.

TAREAS

TODAS SOBRE SU PROYECTO · SI NPM RUN CHECK NO DICE NADA, VAN BIEN

5 tareas, en tres niveles

FÁCIL	**Replicar la clase y tipar una pieza más**: dejen el proyecto como quedó hoy, y agreguen `"mens-watches"` a la unión `Categoria`. Una línea… y después **sigan los subrayados** hasta que se apague el último.
INTERMEDIA	**El estado de la carga, como tipo**: escriban `EstadoCarga` con los cuatro que la tienda ya tiene y háganlo un dato en `main.ts`. Pista para ir más lejos: `never` y comprobación de exhaustividad.
INTERMEDIA	**El campo que falta en el producto**: traigan la descripción de la API como campo **opcional** y muéstrenla recortada. El compilador no los va a dejar usarla sin preguntar. **Sin usar el `!`**.
DIFÍCIL	**"Mis pedidos", con su aduana**: guardar en `techcart_pedidos`, leerlo con unknown y comprobar, y pintar la lista. El nombre del cliente va por `textContent`.
DIFÍCIL	**El detalle de un producto**: pedir uno solo a la API, con su **propia guarda** porque la respuesta tiene otra forma, y sus estados de carga y error. Es lo más parecido a lo que harán en React.

Siguiente bloque · repositorio propio, clase 1 de React

Y ahora sí: React

```
interface PropsTarjeta {
  producto: Producto
  destacada?: boolean
}

export default function Tarjeta({ producto, destacada }: PropsTarjeta) {
  const [abierta, setAbierta] = useState<boolean>(false)
  return <article>{producto.nombre}</article>
}
```

Lo **celeste** ya lo saben leer entero: la interfaz, la prop opcional con su interrogación, el genérico. Lo único genuinamente nuevo es lo gris: que una función **devuelva etiquetas**. Eso se llama **JSX**.

Nosotros escribíamos pintarCatalogo, pintarCarrito, pintarFiltros… y había que acordarse de llamarlas. React hace eso y casi nada más: cambias el dato, él repinta.